

FormBot - Design and Implementation of a Retrieval-augmented Generation (RAG) Module for Automated Assistance in SBI Bank Form Filling

Project Report

Submitted by

Jishnu Prasad

Summer Intern

Summer Internship 2026, SPIRE Lab, IISc, Bangalore

Under the guidance of

Dr. Prasanta Kumar Ghosh

Mentor

Amartyaveer

Department of Electrical Engineering

SPIRE Lab, IISc

Bangalore, Karnataka

July, 2026

Contents

1	Abstract	3
2	Introduction	3
3	Problem Statement	3
4	System Architecture and Methodology	3
4.1	High-Level Architecture	3
4.2	Data and Corpus	4
4.3	Methodology	4
4.4	Technology Stack Evolution	4
4.5	Architecture Modes and Feature Flags	5
4.6	Agentic Retrieval Pipeline	5
5	Development Chronology	5
6	Daily Progress Log	6
7	Evaluation Framework	7
7.1	LLM-as-a-Judge Metrics	7
7.2	Retrieval Metrics	7
7.3	Performance Summary	8
8	Module-by-Module Implementation	8
8.1	Ingestion	8
8.2	Vector Retrieval	8
8.3	Knowledge Graph	9
8.4	Reranking and Context Assembly	9
8.5	Query Handling	9
8.6	Generation	9
8.7	Evaluation	9
8.8	Frontend and Reliability Enhancements	10

9 Results	10
9.1 Key Runs (Chronological)	10
9.2 Evaluation Figures	15
10 Roadmap and Implementation Goals	15
10.1 Goal 1: Canonical Government Schemes Data Platform	15
10.2 Goal 2: Intelligent Upload and Strategy Selection	15
10.3 Goal 3: ChromaDB Vector Store and Embedding Infrastructure	15
10.4 Goal 4: Metadata-First Storage	15
10.5 Goal 5: Document Registry Service	15
10.6 Goal 6: TableRAG for CSV and Excel	15
10.7 Goal 7: Hierarchical Structure-Aware RAG	15
10.8 Goal 8: Agentic Retrieval Architecture	16
10.9 Goal 9: Hybrid Retrieval Layer	16
10.10 Goal 10: Reranking and Context Optimization	16
10.11 Implementation Milestones	16
11 Live Multimodal Extension	16
12 Conclusion and Future Scope	16
13 Acknowledgements	17
14 References	17

1 Abstract

FormBot is a Retrieval-augmented generation (RAG) system that answers questions about State Bank of India (SBI) forms with strictly grounded evidence. The work began with a 206-question manual dataset, expanded with ~1,100 LLM-generated questions to stress-test retrieval and generation. The system evolved from a nearest-neighbour baseline to a hybrid pipeline combining a Neo4j knowledge graph, ChromaDB vector search, early-stage Elasticsearch/BM25 experiments, and cross-encoder reranking, all measured with an LLM-as-judge evaluation harness. The final design delivers grounded answers, live multimodal support (voice and vision), and reproducible evaluation across iterations. Retrieval quality remained the dominant bottleneck; metadata quality, hybrid search, and reranking produced the largest gains.

2 Introduction

SBI forms are dense, procedural, and varied across products. FormBot was created to help applicants and staff get precise answers grounded in official documents while accommodating scanned forms, tables, and scraped text. Source code: github.com/Jishnu-Prasad888/FormBot-Agent-ic-RAG.

3 Problem Statement

- **User challenges:** Scattered procedural details, inconsistent form structures, hard-to-find eligibility rules and document checklists, and multilingual/scanned inputs.
- **Technical challenges:** Retrieval across mixed modalities (PDF, tables, text), reducing hallucinations, avoiding scheme drift, and keeping answers tightly grounded.
- **Constraints:** Banking answers must cite evidence; when evidence is missing, the assistant must refuse or return partial, caveated responses.
- **Objectives:** Deliver a retrieval-grounded assistant that is precise, refusal-capable, and evaluation-backed, with hybrid search (vector + graph + sparse), reranking, and reproducible metrics.

4 System Architecture and Methodology

4.1 High-Level Architecture

- Document ingestion into ChromaDB collections (forms, metadata tables, FAQs, compiled web text) with rich metadata.
- Dual retrieval: dense vector search plus a Neo4j knowledge graph; earlier stages also used Elasticsearch/BM25.

- Cross-encoder reranking and form-aware boosting to assemble the final context window.
- Generation via GPT-4o-mini under a strict grounding prompt.
- Evaluation harness with LLM-as-judge metrics and diagnostic retrieval signals.

4.2 Data and Corpus

- 29 core SBI forms (account opening/closure, nomination, SSA, SCSS, PPF, PMMY, savings products).
- Structured metadata table (eligibility, beneficiary type, required documents, mode of application, offering details).
- Glossary and abbreviations (KYC, NEFT, PMJDY, CIF, CKYC, etc.).
- Evaluation: 206 manual Q&A + ~1,100 LLM-generated Q&A; later runs used 756-pair subsets.
- Primary corpus of ~735 SBI banking JSON/TXT/PDF/Markdown files; Elasticsearch is optional and not required to serve data.

4.3 Methodology

The project followed a diagnose-and-fix loop: run full evaluations, classify failures (retrieval vs. generation vs. corpus gaps), design targeted fixes, and re-run. Metrics were revised when judge behaviour was noisy. Retrieval changes (metadata, hybrid search, reranking) consistently produced the largest gains.

4.4 Technology Stack Evolution

Component	Evolution
LLM & Embeddings	Llama 3.1 (Ollama) -> GPT-4o-mini + text-embedding-3-small/large -> Qwen (fine-tuning experiments for evaluator/retriever).
Vector Store	ChromaDB -> Qdrant -> ChromaDB (for compact experiments).
Keyword Search	None -> Elasticsearch (BM25).
Structured Knowledge	None -> Neo4j knowledge graph (forms, fields, regulations).
Relational Store	SQLite -> PostgreSQL.
Retrieval Fusion	Deduplication -> Reciprocal Rank Fusion.
Reranking	None -> cross-encoder -> two-pass cross-encoder with neighbour expansion.
Evaluation	Cosine heuristics -> LLM-as-judge (5 metrics) + ranking metrics (MRR, Recall@K, nDCG@10).
Interaction Mode	Batch only -> live chat -> live voice + camera OCR.

4.5 Architecture Modes and Feature Flags

The system operated in two main modes: (1) a lightweight JSON+NumPy cosine RAG with 8 backend files, later re-augmented with persistent Chroma at `backend/data/chroma/`, multi-file ingestion, and progress UI (GPT-4o-mini + Ollama supported); and (2) a KAG-first stack with Qdrant as the primary vector store, Neo4j/PostgreSQL graph, feature-flagged components, updated health checks, UUID chunk IDs, and Postgres on port 7000, with Chroma retained as a fallback.

4.6 Agentic Retrieval Pipeline

1. Intent detection for section and product hints.
2. Knowledge graph retrieval for candidate document IDs and related context.
3. Hybrid search with Qdrant/ChromaDB (dense) plus Elasticsearch/BM25 (sparse) fused via RRF.
4. Two-pass cross-encoder reranking with neighbour expansion, then final top- k selection.
5. Iterative Elasticsearch enhancement when context is insufficient.
6. Answer generation under a strict banking-scoped prompt.

Every evaluation query in the KAG mode first pulls Neo4j candidates (seeded automatically from uploads when metadata is present), then hybrid retrieval; a KagGuide frontend view exposes this flow. Metadata-aware RRF combines dense/sparse scores with filters on `product_name`, `category`, `file_name`, `section`, and `chunk_index`, plus query expansion and neighbour expansion for recall.

5 Development Chronology

Date	Commit	Milestone
2026-05-25	247931b	Initial Ollama RAG prototype (FastAPI, SQLite, ChromaDB).
2026-05-25	6c73b78	Switched generation/embeddings to OpenAI; added LLM-as-judge evaluator.
2026-05-29	7cab83d	New SBI banking assistant prompt with retrieval-aware rules.
2026-06-02	424678b	Multi-agent evaluation pipeline (Coordinator, Vector, SQLite, Web, Evaluator).
2026-06-08	4516739..ae6d31d	Query expansion and cross-encoder reranking added.
2026-06-09	cac51ef	Higher retrieval recall; banking-grounded answer generation.
2026-06-10	158cc72	Broad RAG improvements and evaluation metrics.
2026-06-11	2860298..ed9088b	Simplified to nearest-neighbour to debug accuracy regressions.
2026-06-12	4a27f50..c6aa3d1	Web crawler + Elasticsearch ingestion of ~600 SBI pages.

Date	Commit	Milestone
2026-06-15	8887101	RRF, intent detection, metadata filters, neighbour expansion, two-pass rerank.
2026-06-19	71e3b35	Added Neo4j, Qdrant, PostgreSQL (start of KAG).
2026-06-22	038b067..8d7f007	GPT-5.1 experiments and cleanup.
2026-06-23	fa92da6	KAG pipeline reached 68.3% overall with 100% retrieval metrics.
2026-06-29	633da70	Removed Elasticsearch dependency; consolidated requirements.
2026-07-01 to 2026-07-03	9052bf2..f024c99	Reset to cosine baseline; compared Qwen vs. OpenAI for retrieval/eval.
2026-07-07	4f4f1af	Image OCR support (OpenAI + Google GenAI).
2026-07-08	962a0d5	Live user interaction endpoints.
2026-07-10	79c5e2d	Live call and voice conversation (Whisper + TTS).
2026-07-10	d7a516a	Final cleanup for production app.

Table 2: Key milestones across branches.

6 Daily Progress Log

Date	Work Done
20/5/2026	Completed manual Q&A for 14 forms; started LLM-generated pairs.
21/5/2026	Finished LLM Q&A generation; translated for Kannada forms.
22/5/2026	Built first RAG backend/frontend pass.
25/5/2026	Full RAG app with automated CSV eval; ran 206 English manual questions; tested Kannada OCR modules.
26/5/2026	Collected more SBI data (68/206 cells); fixed mismatched eval entries; prompt tuning.
02/6/2026	Agentic RAG; overall score ~50%.
03/6/2026	Nearest-neighbour simplification; added scraped data; score 60.4%.
04/6/2026	Reranking stalled; pruning irrelevant eval questions lifted accuracy to 62%.
05/6/2026	Added query expansion and stronger reranker; score dipped to 57% (prompt issues).
08/6/2026	Cross-encoder + hybrid RAG improved faithfulness/context precision; accuracy 60.4%.
09/6/2026	Retrieval-focused metrics (MRR, Recall@10); measured accuracy at 48%.
10/6/2026	New chunking; simplified to nearest-neighbour to isolate retrieval errors.
11/6/2026	Per-question logging; new ES hybrid after BM25 zero hits; score 61.7%.
12/6/2026	Scraped ~600 pages; ES + ChromaDB; accuracy 63.6%.

Date	Work Done
15/6/2026	Replaced ChromaDB with Elasticsearch for keyword + vector; began Qwen3-8B feedback loop.
16/6/2026	Continued Qwen fine-tuning setup; ES + prompt engineering.
17/6/2026	Let LLM trigger Elasticsearch when context thin; reindexed with richer metadata.
18/6/2026	Shifted to KAG with Neo4j and PostgreSQL.
19/6/2026	Built Neo4j KG; accuracy ~40%.
22/6/2026	Hybrid KAG + RAG (Neo4j, PostgreSQL, Qdrant, ES); score ~57%.
23/6/2026	Analysed eval results; tightened prompt with worked examples.
24/6/2026	Prepared fine-tuning examples for Qwen.
25/6/2026	Swapped Qwen3-14B for Qwen2.5-7B-Instruct to fix fine-tuning errors.
30/6/2026	Debugged CUDA OOM; attempted CPU-only fine-tuning.
01/7/2026	Fine-tuning script hit segmentation faults.
02/7/2026	Traced fine-tuning errors to PyTorch version mismatch.
03/7/2026	Built improved fine-tuning set; cleaned ASR dataset.
06/7/2026	OCR frontend/backend integration.
07/7/2026	Finished backend OCR; started ASR app DB and OCR eval-set script.

Table 3: Daily activity (times omitted).

7 Evaluation Framework

7.1 LLM-as-a-Judge Metrics

- Accuracy (LLM) – semantic match to the expected answer.
- Faithfulness – grounded in retrieved context.
- Context Precision – fraction of retrieved chunks that are relevant.
- Context Recall – whether retrieved context contains all needed facts.
- Answer Relevancy – whether the answer addresses the question.

7.2 Retrieval Metrics

Recall@10/20/50, Mean Reciprocal Rank, and nDCG@10 to isolate retrieval quality from generation quality.

Stage	Overall score
Nearest-neighbour baseline	42%
+ Query expansion (max 2 retries)	47%
+ Cross-encoder reranking (initial)	27–60.4% (unstable)
Elasticsearch + ChromaDB hybrid	61.7–63.6%
Early Neo4j knowledge graph only	~40%
Hybrid KAG (Neo4j + Postgres + Qdrant + ES)	~57%
Full KAG pipeline (best run)	68.3%

Table 4: Overall scores across major iterations.

7.3 Performance Summary

Two lessons emerged: (1) richer metrics can lower measured scores by exposing failure modes; (2) retrieval architecture changes drive accuracy more than swapping the generation model.

8 Module-by-Module Implementation

8.1 Ingestion

- PDF text extraction and chunking (~1,500 characters per chunk); filename links to metadata rows.
- ChromaDB collections: forms, metadata tables, FAQs, compiled third-party text.
- Upload tooling: `upload_documents.py` supports recursive folder ingest, dual progress bars, retry/timeout options, per-run retry toggle, env-configurable base URL, and failure logs.
- Metadata schema: common keys such as `document_id`, `filename`, `chunk_index`, `category`, `product_name`, `section`, `file_name`, `content_hash`, plus PDF/Markdown/Table-specific positions (page, table IDs, parent chunk) to support graph seeding and filtering.

8.2 Vector Retrieval

- Embeddings: OpenAI `text-embedding-3-small` for both corpus and queries.
- Fan-out across collections; merge top candidates before reranking.
- Qdrant HNSW index (unsupported `ef` removed) is the primary store with UUID chunk IDs; Chroma remains a fallback for compact runs.

8.3 Knowledge Graph

- Neo4j graph with nodes for Document, Chunk, Topic, Column; edges HAS_CHUNK, BELONGS_TO, HAS_COLUMN.
- Dual-pass lookup: expand by source document and by query concepts; merge, filter, and fall back to document facts when needed.
- Final scale: 1,917 nodes and 1,897 edges.
- In KAG mode, uploads auto-seed Postgres/Neo4j with Form/FormVersion/Field/Dependency/Requirement/Regulation nodes; Neo4j client updated to `AsyncResult.data()` for reliable health checks.

8.4 Reranking and Context Assembly

- Cross-encoder: `cross-encoder/ms-marco-MiniLM-L-6-v2` with form-hint boosting; pluggable rerankers include `BAAI/bge-reranker-base`, `jinaai/jina-reranker-v2-base-multilingual`, and Cohere.
- Assemble top chunks plus graph facts and glossary hits into the final prompt context after hybrid RRF fusion and metadata filters.

8.5 Query Handling

- Rule-based form-specific detection for reporting and gating.
- Query enrichment with form names when present; glossary fast path for definition queries.

8.6 Generation

- Model: GPT-4o-mini, temperature 0.1; backend configuration toggles allow switching defaults to GPT-5.1 or Ollama-hosted `banking-assistant` with matching accuracy scorer.
- Prompt: strict grounding, refuse when evidence is absent, include numeric/procedural details, partial-answer guidance, scheme-drift warning.

8.7 Evaluation

- LLM-judge metrics: context hit, meaning match, generation accuracy; diagnostic signals for source matching and KG contribution.
- Debug dumps for missed questions; failure buckets for wrong-doc, wrong-chunk, or generation issues.
- Pipeline variants: Elasticsearch as a first-class retriever (synonyms → vector → BM25 → ES → RRF → rerank → neighbour expansion), fp16 reranker fallback to safe mode, batch size 4.

- Frontend eval pauses a batch when LLM API errors occur and surfaces the failing question; partial results stay in memory only. A slimmed endpoint returns chat-style `query/response/status` with optional metrics.

8.8 Frontend and Reliability Enhancements

- Navigation trimmed to focus on core flows; dashboard health map fixed to retain Qdrant/Neo4j statuses and Neo4j health false negatives resolved via `AsyncResult.data()`.
- Evaluation UI adds image OCR upload/server sample buttons and hook-order fix; batches halt cleanly on API errors.
- Live Meet adds camera/screen-share paper detection, PiP, device selection, and throttled OCR with persistent context; streaming coroutine stability improved.
- OCR tooling: `helpfull scripts/check_OCR.py` now supports responses-to-chat fallback, PDF-to-image fallback, progress and retry/timeout flags, API-mode selection, and raw OCR printouts for debugging.
- Backend hardening: CORS-safe error middleware, Chroma collection auto-creation, telemetry disabled to avoid upload 500s, and Ollama API compatibility fixes (`/api/chat` vs. `/api/generate`).

9 Results

9.1 Key Runs (Chronological)

Run (title)	Overall score
Cross-encoder reranking with hybrid RAG	60.4%
After cross-entropy and query expansion	61.7%
15th June	60.0%
15th June with Elasticsearch	64.7%
19th June with Elasticsearch, Neo4j, Qdrant, Postgres	64.9%
19th June GPT-5.1 with Elasticsearch, Neo4j, Qdrant, Postgres	61.3%
23rd June after KAG and LLM prompt changes	56.1%
2nd July simple nearest-neighbour (ChromaDB)	68.0%
2nd July ChromaDB + fine-tuned Qwen 2.5 retriever	68.3%
3rd July Qwen retriever + OpenAI generator	67.4%

Table 5: Evaluation snapshots by date/title.



Figure 1: LLM-as-judge dashboard (15th June, overall 60.0%).



Figure 2: LLM-as-judge dashboard (15th June with Elasticsearch, 64.7%).



Figure 3: LLM-as-judge dashboard (19th June hybrid ES+Neo4j+Qdrant, 64.9%).



Figure 4: LLM-as-judge dashboard (19th June GPT-5.1 hybrid, 61.3%).



Figure 5: LLM-as-judge dashboard (23rd June prompt updates, 56.1%).



Figure 6: LLM-as-judge dashboard (2nd July ChromaDB baseline, 68.0%).



Figure 7: LLM-as-judge dashboard (2nd July ChromaDB + fine-tuned Qwen, 68.3%).



Figure 8: LLM-as-judge dashboard (3rd July Qwen retriever + OpenAI generator, 67.4%).

9.2 Evaluation Figures

10 Roadmap and Implementation Goals

10.1 Goal 1: Canonical Government Schemes Data Platform

Normalized scheme data in SQLite with list/filter/detail APIs; indexed on scheme name, state, ministry.

10.2 Goal 2: Intelligent Upload and Strategy Selection

Auto-detect CSV/XLSX/PDF/Markdown/text/audio and route to TableRAG, hierarchical RAG, vector RAG, or speech pipeline; no manual configuration.

10.3 Goal 3: ChromaDB Vector Store and Embedding Infrastructure

Centralized embeddings (Nomic Embed Text V2 MOE via Ollama for offline runs; OpenAI embeddings for online) with incremental/batch indexing, re-indexing, metadata filtering, and multi-collection search.

10.4 Goal 4: Metadata-First Storage

Every chunk stores filename, document ID/type, section, language, chunk IDs, upload date, retrieval strategy; sample record used in experiments included eligibility, subsection, page number, parent chunk, and upload date.

10.5 Goal 5: Document Registry Service

SQLite registry tracking document ID, filename, type, retrieval strategy, language, upload time, chunk count, embedding model, and collection name to support re-index, versioning, deletion, analytics.

10.6 Goal 6: TableRAG for CSV and Excel

Schema index for columns/types/relations plus value index for rows/entities; merges both before generation to preserve table structure.

10.7 Goal 7: Hierarchical Structure-Aware RAG

Preserve document hierarchy (section → subsection → chunk) for PDFs/Markdown; include metadata nodes for retrieval.

10.8 Goal 8: Agentic Retrieval Architecture

Specialized agents: SQLite, Vector Retrieval, Document Routing, Web Enrichment, Retrieval Evaluation, Coordinator. Query expansion example: “PM Kisan Karnataka eligibility” expands to “Pradhan Mantri Kisan Samman Nidhi”, “Farmer Income Support”, “Eligibility Karnataka”. Vector result example: filename *pm_kisan_guidelines.pdf*, section *Eligibility*, score 0.94.

10.9 Goal 9: Hybrid Retrieval Layer

Combine SQL, vector, TableRAG, metadata search, BM25 keyword, and web retrieval; fuse scores, rerank, and assemble final context.

10.10 Goal 10: Reranking and Context Optimization

Retrieve top 50, rerank, keep top 5–10; deduplicate, drop distractors, weight metadata.

10.11 Implementation Milestones

1) Normalize schemes DB and expose API; 2) Build document registry and validation; 3) Implement upload detection/routing; 4) Add TableRAG, hierarchical RAG, vector RAG pipelines; 5) Integrate ChromaDB collections with metadata filters; 6) Add agent orchestration, retrieval evaluation, reranking, web enrichment; 7) Ship frontend views for schemes, documents, sources, answers.

11 Live Multimodal Extension

The final phase extended FormBot to voice and vision: Whisper-1 speech-to-text for voice queries, OpenAI TTS for spoken responses, live camera scanning with pHash throttling to feed GPT-4o/Gemini for field extraction, and stateful session tracking for evolving form values. OCR providers are switchable (Gemini/OpenAI/Ollama), with OpenAI set as default after experimentation. An ASR audio dataset was collected via the Read with Spire app with curated prompts and supervised data collection.

12 Conclusion and Future Scope

FormBot evolved from a nearest-neighbour baseline to a knowledge-augmented, multi-agent RAG system with live multimodal support. Retrieval quality, not model choice, dominated accuracy: better metadata, hybrid search, and reranking moved scores from the low 40s to ~68%. Next steps include fine-tuning an open-weight judge and retriever, expanding Kannada OCR and QA coverage, stabilising ranking-metric evaluation, and deepening OCR/ASR so scanned and spoken inputs perform comparably to text.

13 Acknowledgements

Thanks to SPIRE Lab, IISc for infrastructure and mentorship; Dr. Prasanta Kumar Ghosh for enabling this work; Supriya R. for support on ASR data collection; Amartya Veer and Ananya for helping with the implementation and swiftly resolving doubts.

14 References

1. P. Lewis et al., “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” NeurIPS, 2020.
2. V. Karpukhin et al., “Dense Passage Retrieval for Open-Domain Question Answering,” EMNLP, 2020.
3. S. Robertson and H. Zaragoza, “The Probabilistic Relevance Framework: BM25 and Beyond,” Foundations and Trends in Information Retrieval, 2009.
4. O. Khattab and M. Zaharia, “ColBERT: Efficient and Effective Passage Search via Contextualized Late Interaction over BERT,” SIGIR, 2020.
5. G. Izacard and E. Grave, “Leveraging Passage Retrieval with Generative Models for Open Domain Question Answering,” EACL, 2021.
6. Y. Gao et al., “Retrieval-Augmented Generation for Large Language Models: A Survey,” arXiv:2312.10997, 2023.
7. N. Reimers and I. Gurevych, “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks,” EMNLP-IJCNLP, 2019.
8. B. Peng et al., “Check Your Facts and Try Again: Improving Large Language Models with External Knowledge and Automated Feedback,” arXiv:2302.12813, 2023.
9. S. E. Robertson and K. Spärck Jones, “Relevance Weighting of Search Terms,” JASIS, 1976.